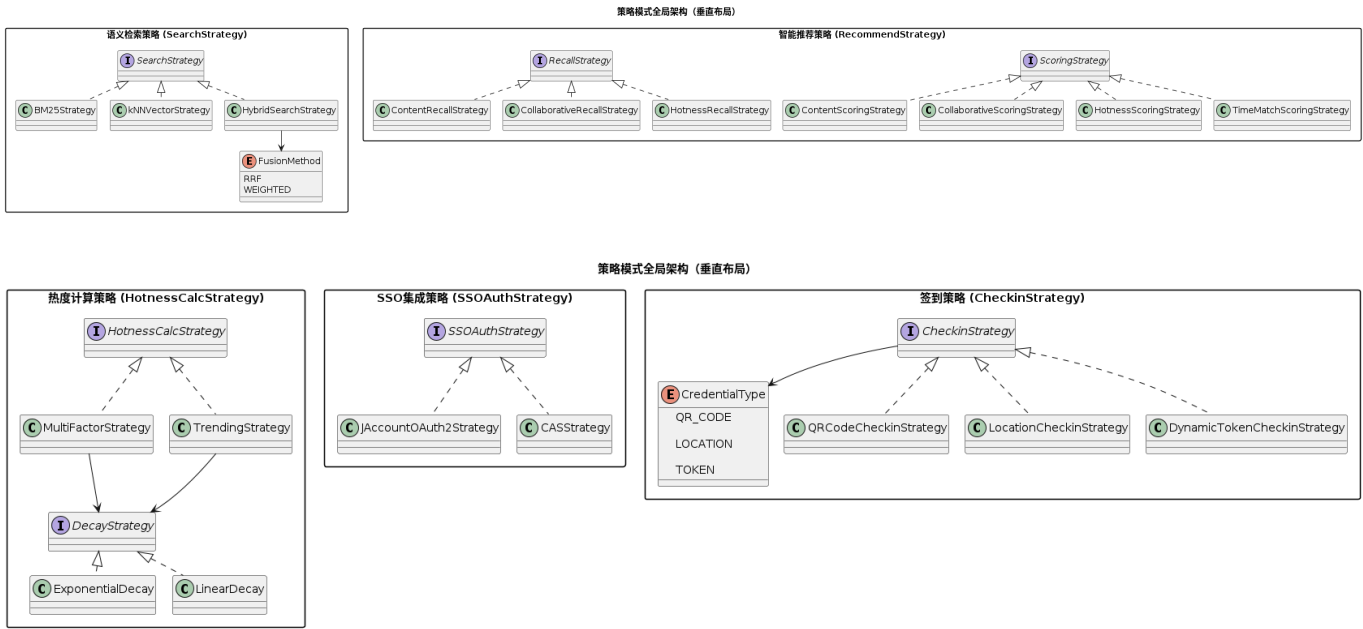


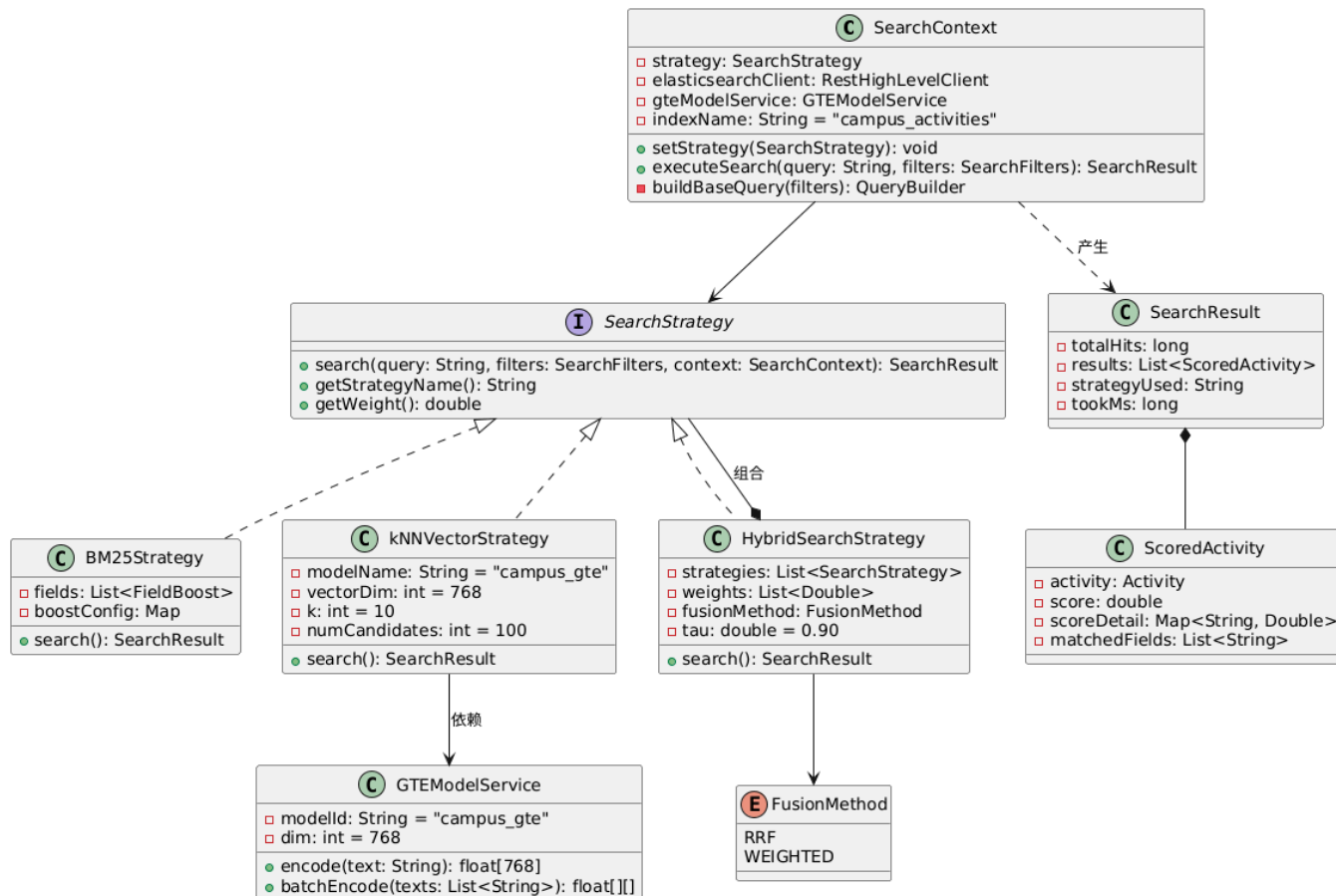
校园活动一站式服务平台的设计模型

设计模型覆盖的模块包括语义检索、智能推荐、SSO集成、签到、智能分析、社区聚类、热度计算这七大模块。它在平台中封装五类算法族，每类算法族对应一个策略接口，这是它的整体架构：



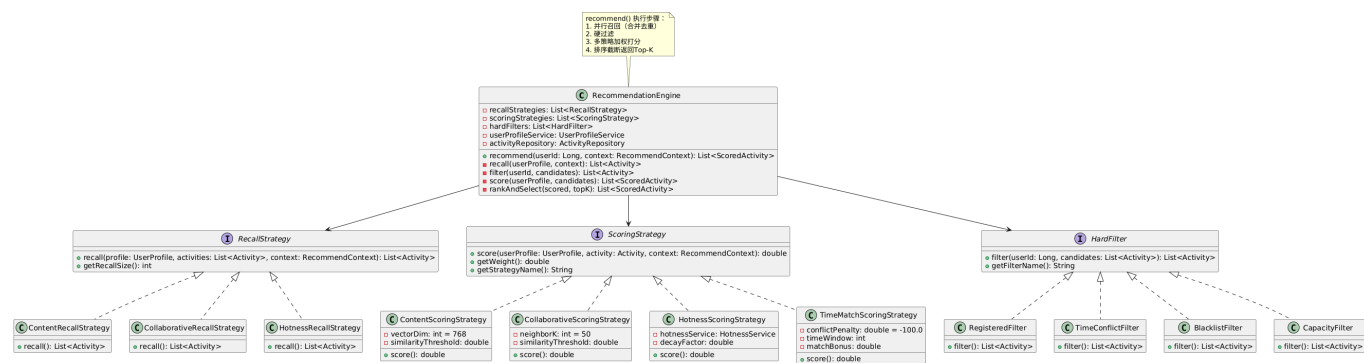
语义检索模块的设计模型

语义检索需要支持三种检索模式：关键词精确匹配（BM25）、语义向量检索（kNN）、以及两者的混合。系统采用混合检索模式（BM25 || kNN， $\tau=0.90$ ），检索策略需要灵活切换以适应不同搜索场景。它的设计模型如下：



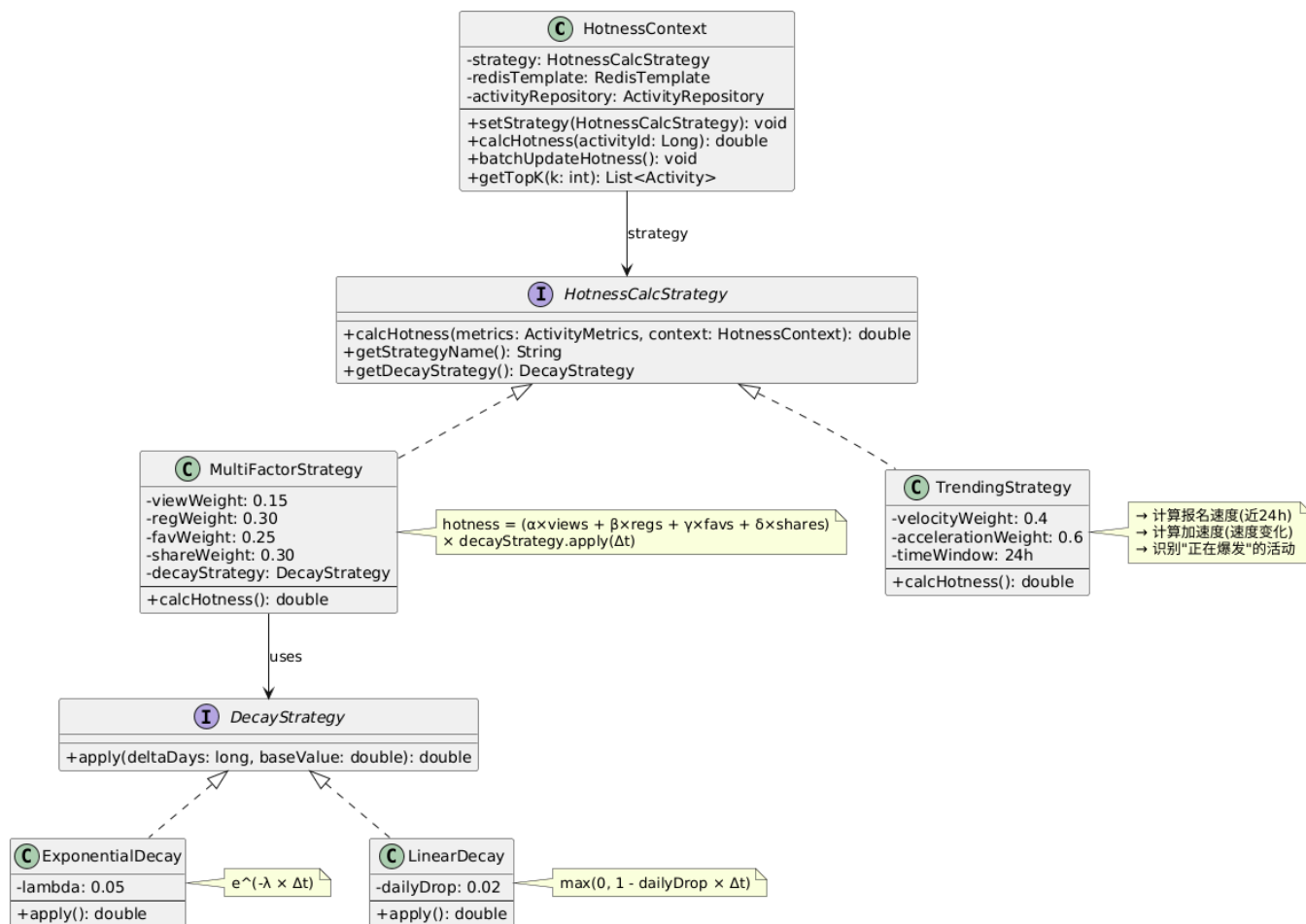
智能推荐模块的设计模型

推荐系统需要实现"喜好向量kNN + 硬过滤 + 社交/热度/时间加权"的综合推荐流程。这里包含召回策略和打分策略。它的设计模型如下：



热度计算模块的设计模型

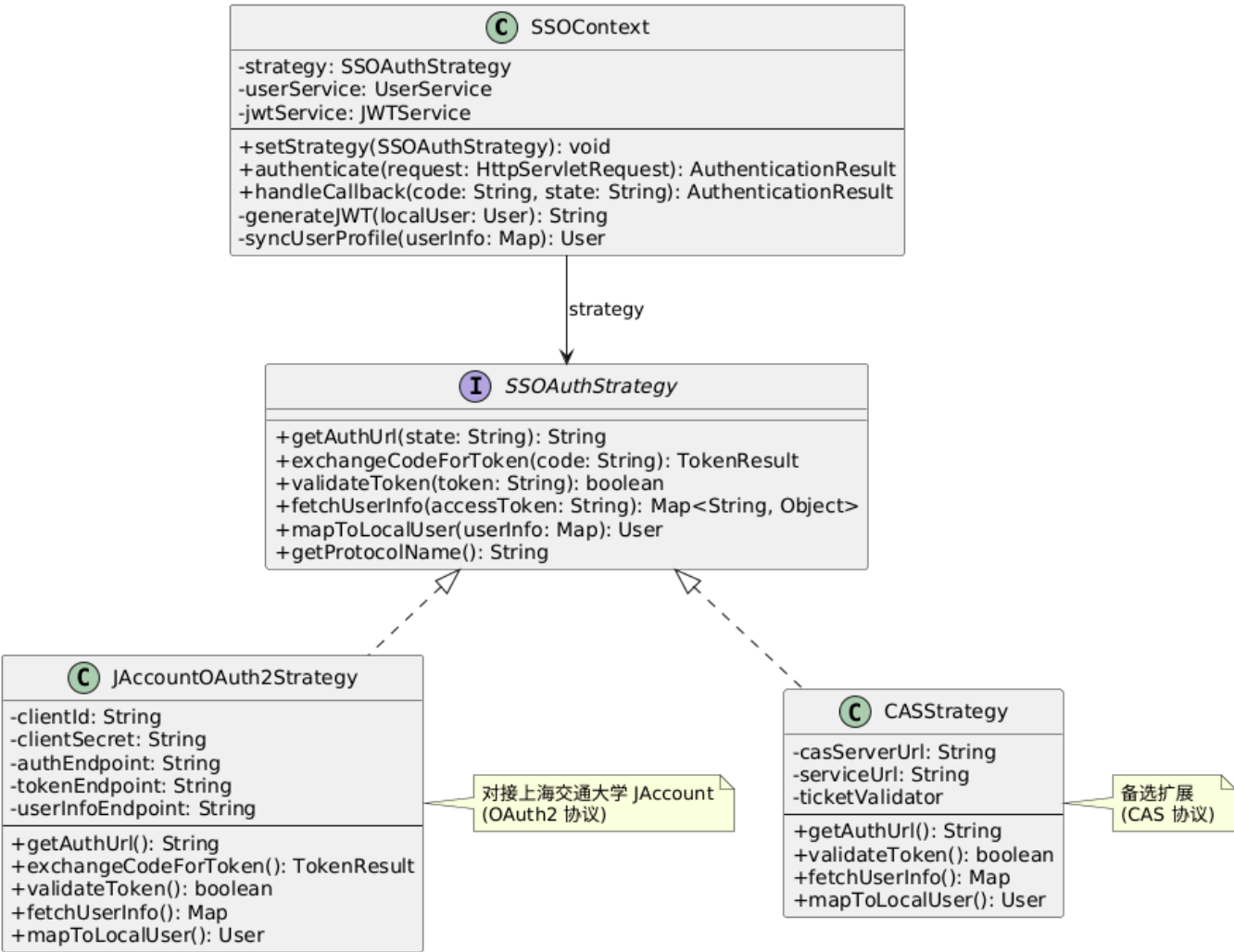
热度计算需要综合浏览量、报名人数、收藏数、分享次数及增长速度等多个因子，并采用指数衰减策略确保近期活动获得更高权重。它的设计模型如下：



日常使用 MultiFactorStrategy，活动页使用 TrendingStrategy 识别爆发活动，衰减策略独立接口，支持指数衰减和线性衰减切换，通过Redis缓存定时(每5分钟)批量更新热度值，最终的热度值同步至Elasticsearch索引，支持按热度排序检索。

SSO集成模块的设计模型

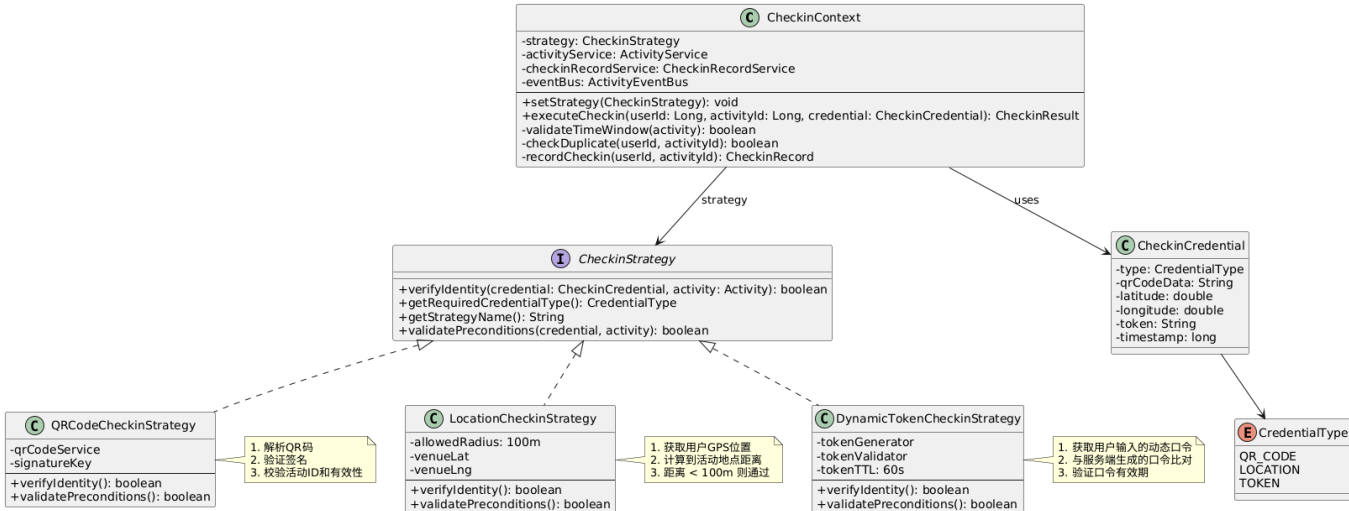
SSO集成需要支持与上海交通大学JAccount（OAuth2协议）对接，未来可能扩展到CAS协议或其他学校认证系统。不同协议在Token验证、用户信息获取上存在差异，但认证结果的生成是一致的。它的设计模型如下：



新增CAS协议只需实现SSOAuthStrategy接口，系统将自动完成JAccount与本地账号的创建与绑定。登录态使用 JWT 令牌管理，支持跨域和移动端。

签到模块的设计模型

平台支持三种签到方式：二维码扫描、GPS定位验证、动态口令验证。三种方式在身份验证逻辑上不同，但签到记录、时间窗口校验等是相同的。它的设计模型如下：

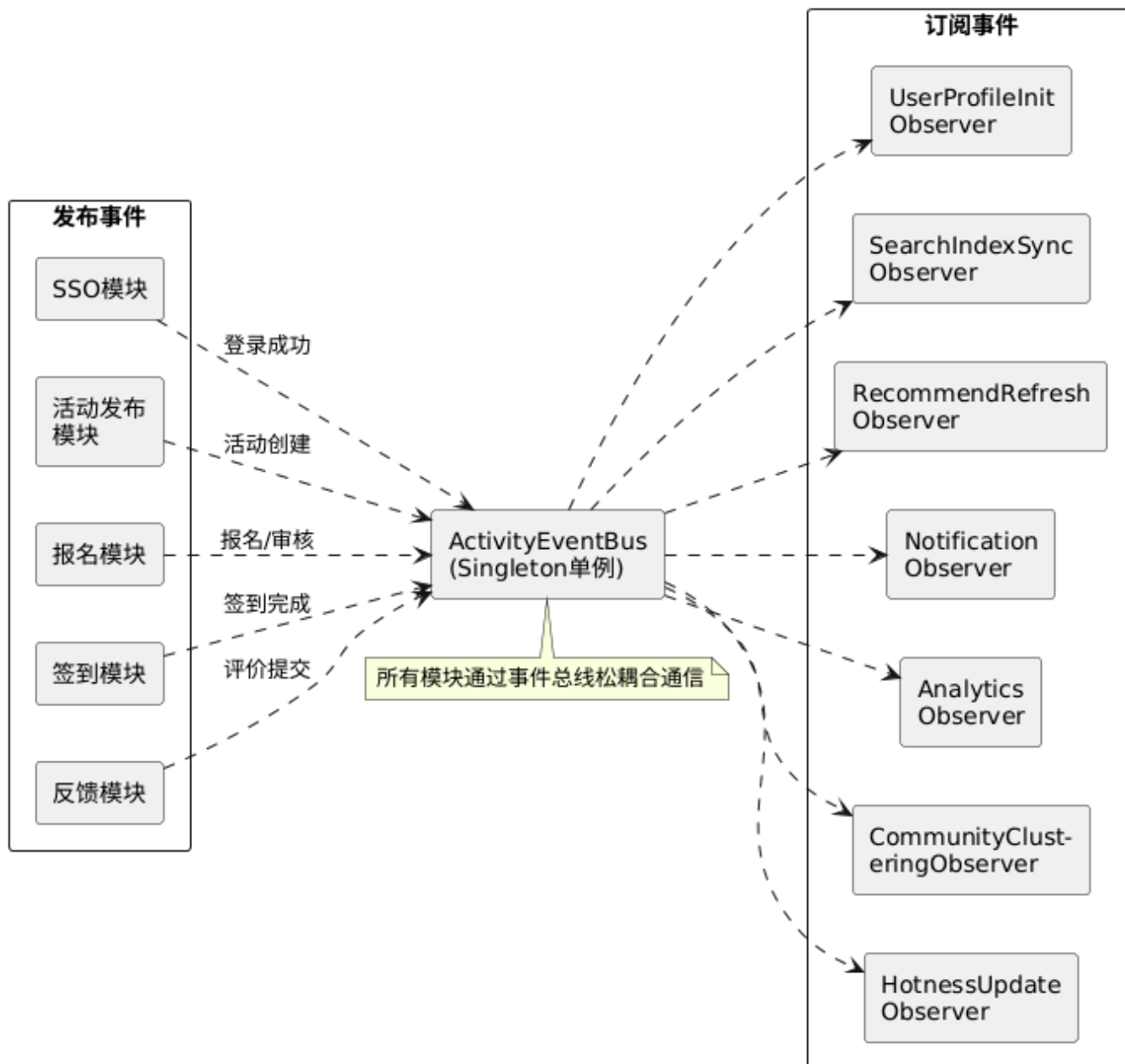


签到流程如下：

1. `validateTimeWindow(activity)`: 检查当前时间是否在活动签到时间窗内。如果不在窗内, 则返回 `CHECKIN_TIME_INVALID`。
2. `checkDuplicate(userId, activityId)`: 查询用户是否已签到。如果已签到, 则返回 `ALREADY_CHECKED_IN`。
3. `strategy.validatePreconditions(credential, activity)`: `QRCode` 用于验证 QR 码未过期, `Location` 用于请求定位权限, 获取GPS数据, `Token` 用于确认用户在有效时间内输入。
4. `strategy.verifyIdentity(credential, activity)`: `QRCode` 用于解析 QR 码签名, 校验活动ID匹配。
`Location` 用于计算距离, 当距离小于 100 米时, 验证通过。`Token` 用于比对用户输入的口令与服务端生成的口令, 如果口令在有效期 60s 内, 则验证通过。
5. `recordCheckin(userId, activityId)`: 写入签到记录到数据库, 并返回 `CheckinRecord`。
6. 发布 `CHECKIN_COMPLETED` 事件, 触发观察者链。

每种签到方式的验证逻辑完全封装在策略类中, 时间窗校验、重复签到检查由Context统一处理, QR码带签名防伪造, 定位校验距离防远程签到, 签到完成后发布事件, 触发分析、热度等模块。

在我们的设计模型中, 我们还构建了平台的统一事件驱动机制, 覆盖所有模块的状态联动:



C

ActivityEvent

□

eventId: String // 事件唯一ID

□

eventType: EventType // 事件类型

□

userId: Long // 触发用户ID

□

activityId: Long // 关联活动ID (可为空)

□

timestamp: LocalDateTime // 事件时间

□

payload: Map<String, Object> // 事件携带数据

□

source: String // 事件来源模块

E

«enumeration»
EventType

USER_LOGIN_SUCCESS // 用户登录成功

USER_PROFILE_UPDATED // 用户档案更新

ACTIVITY_CREATED // 活动创建

ACTIVITY_PUBLISHED // 活动发布上线

ACTIVITY_UPDATED // 活动信息更新

ACTIVITY_CANCELLED // 活动取消

ACTIVITY_STARTED // 活动开始

ACTIVITY_ENDED // 活动结束

USER_REGISTERED // 用户报名

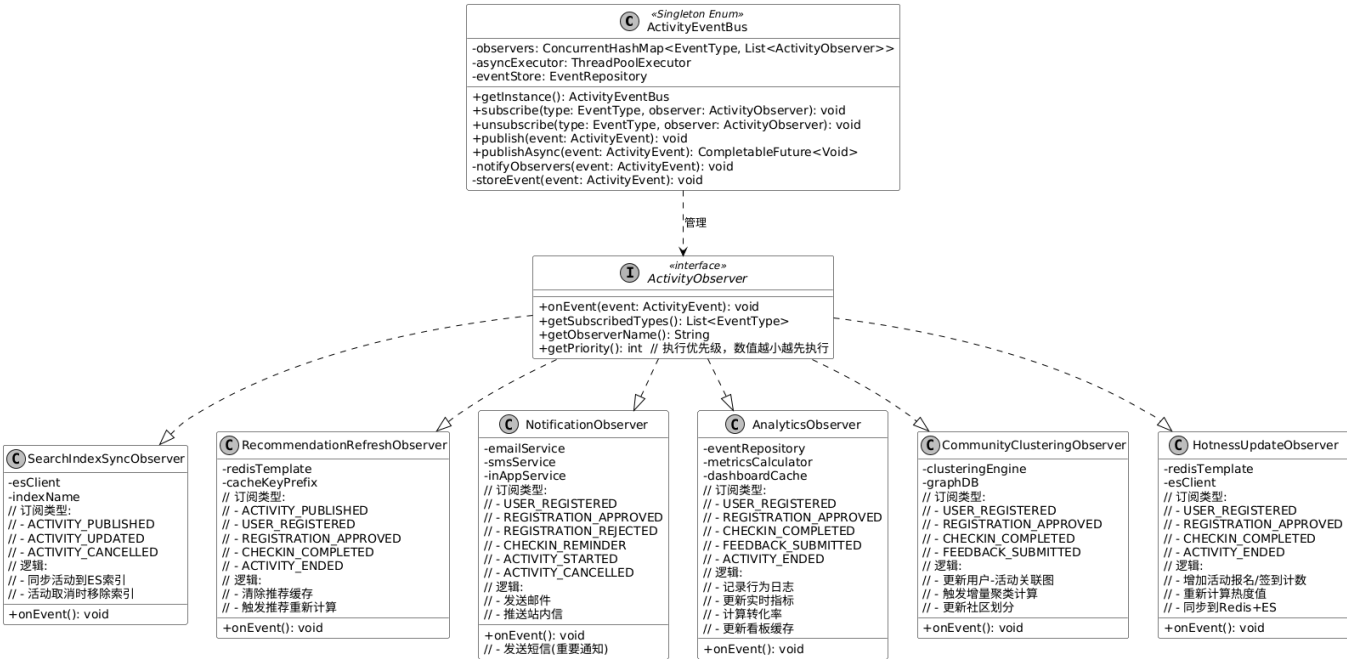
REGISTRATION_APPROVED // 报名审核通过

REGISTRATION_REJECTED // 报名被拒绝

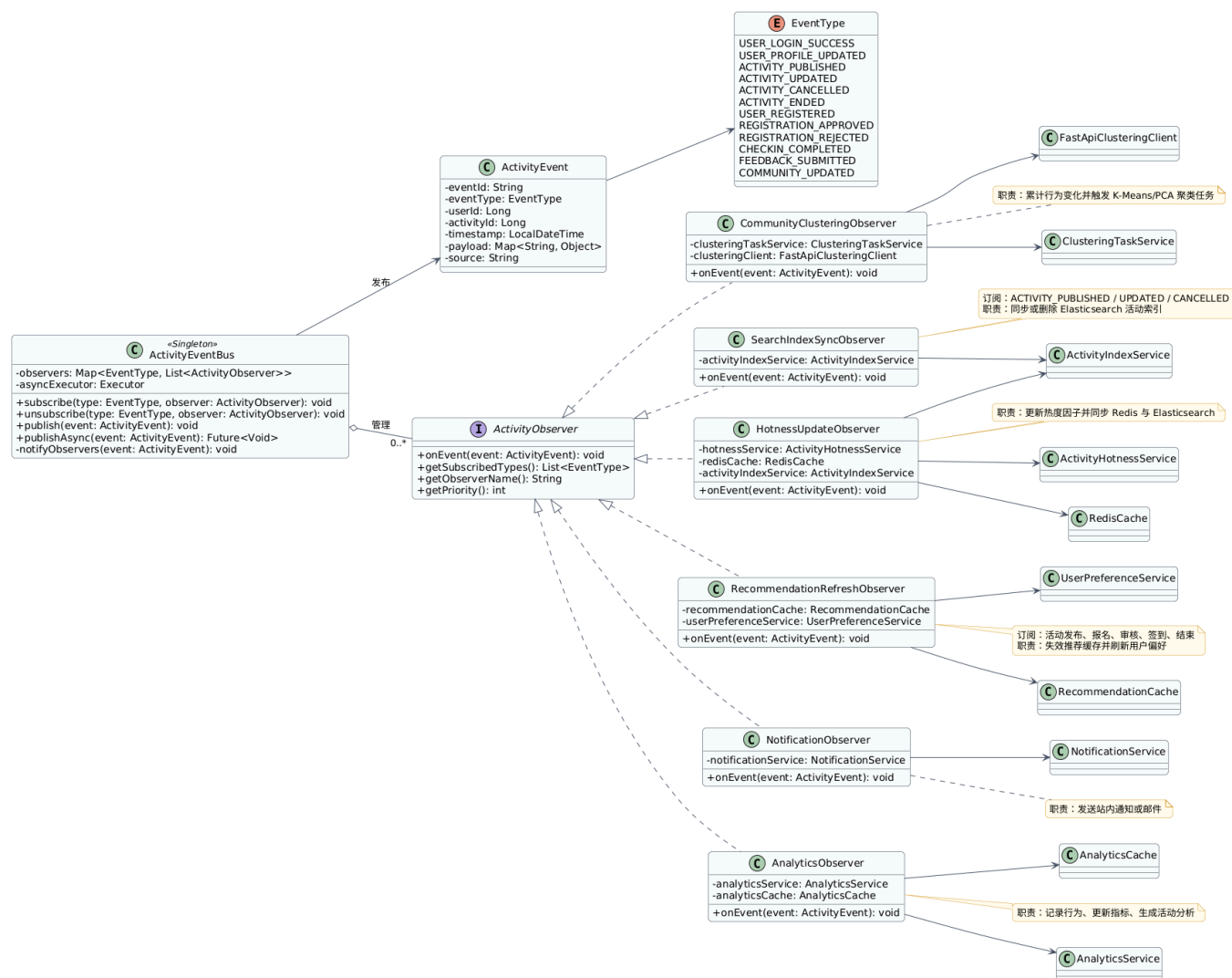
CHECKIN_COMPLETED // 签到完成

FEEDBACK_SUBMITTED // 评价提交

COMMUNITY_UPDATED // 社区聚类结果更新



3.4 各 Observer 详细设计——观察者模式类图



以下是对各个模块的详细说明：

- SearchIndexSyncObserver（语义检索模块）：该模块订阅的事件为 ACTIVITY_PUBLISHED, ACTIVITY_UPDATED, ACTIVITY_CANCELLED，它需要保证搜索数据的实时性，每当活动发布或更新时，都需要同步到ES索引，活动取消时，从索引中删除。
- RecommendationRefreshObserver（智能推荐模块）：该模块订阅的事件为 ACTIVITY_PUBLISHED, USER_REGISTERED, REGISTRATION_APPROVED, CHECKIN_COMPLETED, ACTIVITY_ENDED，它的作用是使相关用户推荐缓存实效，并触发重新计算。
- NotificationObserver（全模块通知）：该模块订阅的事件为 USER_REGISTERED, REGISTRATION_APPROVED, REGISTRATION_REJECTED, CHECKIN_REMINDER, ACTIVITY_STARTED, ACTIVITY_CANCELLED, ACTIVITY_ENDED，它负责根据事件类型向相关用户发送通知。
- AnalyticsObserver（智能分析模块）：该模块订阅的事件为 USER_REGISTERED, REGISTRATION_APPROVED, CHECKIN_COMPLETED, FEEDBACK_SUBMITTED, ACTIVITY_ENDED，它负责记录所有行为日志，实时计算分析指标。
- CommunityClusteringObserver（社区聚类模块）：该模块订阅的事件为 USER_REGISTERED, REGISTRATION_APPROVED, CHECKIN_COMPLETED, FEEDBACK_SUBMITTED，它负责维护用户-活动二分图，增量更新社区划分。

- HotnessUpdateObserver（热度计算模块）：该模块订阅的事件为 USER_REGISTERED, REGISTRATION_APPROVED, CHECKIN_COMPLETED, ACTIVITY_ENDED，它负责更新活动热度相关计数，触发热度重算。

这是设计模型中各个事件的联动图：

